

Transactions Data Quality Profile

Dataset: transactions.csv — 20,420 rows x 16 columns (synthetic e-commerce / card transactions)
Author: [YOUR NAME] | **Course / Team:** [COURSE OR TEAM] | **Date:** [DATE] **Tooling:** Python 3.10, pandas, numpy, matplotlib, seaborn (PySpark equivalent included in Section 9)

Executive summary

This report profiles the transactions.csv dataset along seven data-quality dimensions — schema, completeness, uniqueness, distribution, outliers, consistency and correlation structure — and closes with eight executable rules that can be re-run on new loads of the same table.

The table is **usable but not production-clean**. Six of the eight quality rules fail. The most material issues, in order of impact:

#	Finding	Magnitude	Severity
1	transaction_id is not unique — the declared primary key repeats	420 repeated IDs (840 rows involved, 4.11%)	High
2	amount is inconsistent with its own line items (quantity x unit_price x (1 - discount_pct))	1,185 rows, 5.80%	High
3	Exact duplicate rows	400 duplicates, 1.96% of the table	High
4	Negative amount values on rows with no refund convention	26 rows	Medium
5	Impossible customer_age values (min -5, max 999)	16 rows	Medium
6	Missing values concentrated in four columns	discount_pct 7.00%, customer_age 4.98%, country / currency 2.21% each	Medium
7	country / currency missingness is not random — it occurs only in the online channel	4.01% of online rows vs 0.00% in store	Medium
8	amount is severely right-skewed (skew = 51.5), which breaks naive statistical thresholds	mean 258.84 vs median 63.54	Context

Two rules pass: no nulls in the required key fields (customer_id, amount, transaction_ts) and every merchant_category value falls inside the known 8-value domain.

Recommendation in one line: the ID collisions and the amount/line-item mismatch must be fixed upstream before this table is used for any keyed join or revenue aggregation; the skew and the non-random missingness should be disclosed, not silently imputed away.

Scope and method

Each section below states the check performed, the code that performs it, and the finding. Sections follow the standard profiling sequence: schema, completeness, uniqueness, descriptive

statistics, distribution shape, outliers, correlation, distributed equivalent, and finally the rule set that turns the findings into an automated test suite.

The notebook is executed top to bottom; later cells reuse objects (`df`, `lower`, `upper`) created earlier. `transactions.csv` is expected in the same directory as the notebook.

1. Setup and data load

pandas provides the `DataFrame` used throughout; numpy supports the z-score and clipping operations in Sections 6 and 7; matplotlib and seaborn produce the figures. `parse_dates` is applied at load time so `transaction_ts` arrives as a true `datetime64` column rather than text, which is a prerequisite for any later time-based check.

CODE

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")    # nicer default look for all charts below
pd.set_option("display.max_columns", None)    # don't truncate wide DataFrames when printed

DATA_PATH = "transactions.csv"
df = pd.read_csv(DATA_PATH, parse_dates=["transaction_ts"])
df.shape    # expect: (20420, 16)
```

OUTPUT

```
(20420, 16)
```

2. Schema profile

Check. Confirm the physical type of every column against the type its business meaning implies. A date stored as text or a flag stored as a float are schema defects in their own right, and they invalidate every downstream check before it runs.

Finding. All 16 columns load with the expected type: `transaction_ts` as `datetime64[ns]`, six integer columns, four floats, and five `object` (text) columns for the categoricals. No schema defects were found.

`info()` also doubles as a first completeness signal: `customer_age` (19,403), `country` and `currency` (19,968 each) and `discount_pct` (18,991) report non-null counts below the 20,420 row total. Those gaps are quantified in Section 3.

CODE

```
df.info()
```

OUTPUT

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20420 entries, 0 to 20419
Data columns (total 16 columns):
#   Column                Non-Null Count  Dtype
---  -
0   transaction_id         20420 non-null  int64
1   transaction_ts         20420 non-null  datetime64[ns]
2   customer_id           20420 non-null  int64
3   customer_age           19403 non-null  float64
4   account_age_days       20420 non-null  int64
5   merchant_category      20420 non-null  object
6   payment_method         20420 non-null  object
7   channel                20420 non-null  object
8   country                19968 non-null  object
9   currency               19968 non-null  object
10  quantity               20420 non-null  int64
11  unit_price             20420 non-null  float64
12  discount_pct           18991 non-null  float64
13  amount                 20420 non-null  float64
14  num_prior_txns         20420 non-null  int64
15  is_fraud               20420 non-null  int64
dtypes: datetime64[ns](1), float64(4), int64(6), object(5)
memory usage: 2.5+ MB
```

Check. Inspect the first rows for structural sanity, then enumerate the domain (distinct values) of each categorical column. `dropna=False` is used deliberately so that missing values are counted as their own category instead of being hidden.

Finding. The domains are small and well defined: `merchant_category` has 8 values, `payment_method` 4, `channel` 2 (online / in_store), `country` 7 real values plus NaN, and `currency` 3 real values plus NaN. No unexpected or misspelled categories appear. These 8 merchant categories are what Rule 7 in Section 10 validates against.

One structural observation worth noting for later: `currency` carries only 3 distinct values (MXN, BRL, USD) against 7 countries, so `amount` is **not** expressed in a single currency and is not strictly comparable across rows. This report treats `amount` as a raw numeric field, as the dataset provides no FX rates.

CODE

```
display(df.head())

cat_cols = ["merchant_category", "payment_method", "channel", "country", "currency"]
for c in cat_cols:
    print(c, "->", df[c].nunique(dropna=False), "unique values:", df[c].unique())
```

OUTPUT

	transact ion_id	transact ion_ts	custome r_id	custome r_age	account_ age_ days	merchan t_ category	payment_ method	channel	country	currency	quantity	unit_ price	discount_ pct	amount	num_ prior_ txns	is_fraud
0	14920	2025-06-26 02:19:45	12634	47.0	691	travel	credit_ card	online	MX	MXN	4	459.97	0.219	1438.28	30	0
1	3568	2025-02-11 16:46:57	13878	21.0	539	restauran t	debit_ card	in_store	BR	BRL	3	9.46	0.066	26.47	16	0

	transaction_id	transaction_ts	customer_id	customer_age	account_age_days	merchant_category	payment_method	channel	country	currency	quantity	unit_price	discount_pct	amount	num_prior_txns	is_fraud
2	3351	2025-02-12 12:08:39	10117	41.0	2054	electronics	credit_card	in_store	US	USD	2	174.57	0.043	336.19	94	1
3	16021	2025-03-25 05:25:39	13998	47.0	19	grocery	wallet	online	MX	MXN	4	21.19	0.150	69.93	0	0
4	14031	2025-08-11 12:10:20	14956	NaN	179	restaurant	bank_transfer	in_store	BR	BRL	3	6.17	0.148	14.53	11	0

merchant_category -> 8 unique values: ['travel' 'restaurant' 'electronics' 'grocery' 'entertainment' 'utilities' 'healthcare' 'clothing']
payment_method -> 4 unique values: ['credit_card' 'debit_card' 'wallet' 'bank_transfer']
channel -> 2 unique values: ['online' 'in_store']
country -> 8 unique values: ['MX' 'BR' 'US' 'AR' 'CA' nan 'ES' 'CO']
currency -> 4 unique values: ['MXN' 'BRL' 'USD' nan]

3. Completeness (missing values)

Check. Count and rate of nulls per column. The rate matters more than the count: 1,017 missing values is alarming in isolation but manageable at 4.98% of rows, and the percentage is what belongs in a quality report.

Finding. Missingness is confined to four columns:

Column	Missing	Rate
discount_pct	1,429	7.00%
customer_age	1,017	4.98%
country	452	2.21%
currency	452	2.21%

The remaining 12 columns are complete. `country` and `currency` share an identical count, which suggests they go missing together in the same rows rather than independently — consistent with a single upstream enrichment step failing for those records.

CODE

```
null_report = pd.DataFrame({
    "n_missing": df.isna().sum(),
    "pct_missing": (df.isna().mean() * 100).round(2),
}).sort_values("pct_missing", ascending=False)
null_report[null_report.n_missing > 0]
```

OUTPUT

	n_missing	pct_missing
discount_pct	1429	7.00
customer_age	1017	4.98
country	452	2.21
currency	452	2.21

Check. Test whether missingness is random or concentrated in a segment, by cross-tabulating `channel` against the null indicator for `country` and normalising by row.

Finding. The gaps are **not** missing at random: 4.01% of `online` transactions lack a country, against 0.00% of `in_store` transactions.

Why it matters. Under missing-completely-at-random, dropping or imputing affected rows is broadly safe. Here it is not: dropping them would remove only online transactions and silently bias every channel comparison downstream. The correct treatment is to keep the rows, mark the gap explicitly (e.g. an `UNKNOWN` sentinel), and disclose the mechanism — not to impute a modal country that would fabricate geography for one specific channel.

CODE

```
pd.crosstab(df["channel"], df["country"].isna(), normalize="index").rename(columns={True:
"country_missing_rate"})
```

OUTPUT

country	False	country_missing_rate
channel		
in_store	1.000000	0.000000
online	0.959851	0.040149

4. Uniqueness (duplicates)

Two distinct failure modes are checked separately, because they have different causes and different remedies:

1. **Exact duplicate rows** — every column matches another row. Typically a pipeline replaying a record after a timeout that had in fact succeeded. Safe to de-duplicate.
2. **Key collisions** — the same `transaction_id` appears on rows with *different* payloads. Far more serious: the declared primary key is not a key, so any join or aggregation on it silently fans out or double-counts. This cannot be fixed by de-duplication, only upstream.

Finding. 400 exact duplicate rows (1.96% of the table) and 420 repeated `transaction_id` values. Filtering the ID groups down to those where `amount` differs isolates the second, more serious case: pairs sharing an ID while disagreeing on customer, timestamp, category and amount — i.e. genuinely different transactions issued the same identifier. `transaction_id` therefore cannot be trusted as a primary key in its current state.

CODE

```

full_dupes = df[df.duplicated(keep=False)]
print("Exact duplicate rows:", df.duplicated().sum(), f"({df.duplicated().mean()*100:.2f}% of rows)")

id_dupes = df[df.duplicated(subset=['transaction_id'], keep=False)]
print("transaction_id values that repeat:", df['transaction_id'].duplicated().sum())

# the interesting case: same id, different amount -> real data-integrity problem
id_dupes.sort_values('transaction_id').groupby('transaction_id').filter(lambda g:
g['amount'].nunique() > 1).head(10)

```

OUTPUT

```

Exact duplicate rows: 400 (1.96% of rows)
transaction_id values that repeat: 420

```

	transact ion_id	transact ion_ts	custome r_id	custome r_age	account_ age_ days	merchan t_ category	payment_ method	channel	country	currency	quantity	unit_ price	discount_ pct	amount	num_ prior_ txns	is_fraud
20162	151	2025-08-20 16:33:19	14763	77.0	144	restaurant	credit_card	online	MX	MXN	3	16.24	0.081	46.06	5	0
13846	151	2025-05-28 02:46:21	14785	34.0	280	healthcare	debit_card	online	MX	MXN	3	80.49	0.207	260.04	8	1
8921	2843	2025-07-26 22:17:41	14610	27.0	388	restaurant	credit_card	online	CO	USD	3	10.01	NaN	28.42	15	0
2233	2843	2025-03-09 16:05:39	10318	21.0	49	restaurant	debit_card	online	MX	MXN	3	11.05	0.029	45.87	3	0
13903	3178	2025-09-08 07:12:56	10794	51.0	122	electronics	credit_card	online	MX	MXN	4	82.16	0.031	435.69	4	0
17607	3178	2025-03-22 02:42:13	14321	29.0	59	travel	bank_transfer	in_store	MX	MXN	8	506.73	0.445	2249.96	2	1
14951	4005	2025-08-05 21:22:33	10382	34.0	359	travel	wallet	in_store	US	USD	3	354.00	0.324	718.67	19	0
18569	4005	2025-05-21 02:41:30	13460	51.0	4000	travel	credit_card	in_store	MX	MXN	8	100.41	0.192	890.72	172	0
19423	4100	2025-08-19 20:07:07	10852	53.0	577	grocery	debit_card	online	AR	USD	1	7.10	NaN	3.12	18	0
19008	4100	2025-04-01 16:52:48	14662	35.0	1329	entertainment	debit_card	online	US	USD	3	6.59	0.149	16.22	54	0

5. Descriptive statistics

Check. `describe()` over the numeric columns, transposed for readability. Beyond the summary itself, the mean-versus-median comparison per row is the fastest available screen for skew and outlier exposure.

Finding. Three columns show defects visible in the summary alone:

- **amount** — mean 258.84 against a median of 63.54 (roughly 4x), a minimum of **-1,418.43** and a maximum of **125,157.49** against a 75th percentile of 187.39. Negative sale amounts are invalid on their face, and the jump from Q3 to the maximum is three orders of magnitude.
- **customer_age** — minimum **-5** and maximum **999**, both impossible for a human customer. The quartiles (32 / 41 / 50) are entirely plausible, so these are isolated sentinel or data-entry errors rather than a broken column.
- **account_age_days** — maximum of 4,000 days (about 11 years) is high but not impossible; flagged for review rather than as a defect.

is_fraud has a mean of 0.0439, i.e. a 4.4% positive rate — a rare-event class, which is relevant when interpreting any correlation involving it (Section 11).

CODE

```
df.describe(include="number").T
```

OUTPUT

	count	mean	std	min	25%	50%	75%	max
transaction_id	20420.0	10000.741871	5769.992727	1.00	4997.7500	10006.500	14994.25	20000.00
customer_id	20420.0	12503.701371	1439.504185	10000.00	11262.0000	12500.000	13753.00	14999.00
customer_age	19403.0	41.341597	21.050714	-5.00	32.0000	41.000	50.00	999.00
account_age_days	20420.0	400.090597	400.483627	1.00	118.0000	279.000	553.00	4000.00
quantity	20420.0	3.202106	1.480846	1.00	2.0000	3.000	4.00	11.00
unit_price	20420.0	77.466113	138.951222	1.15	12.2575	25.105	68.83	2077.53
discount_pct	18991.0	0.157950	0.112017	0.00	0.0710	0.134	0.22	0.60
amount	20420.0	258.842052	1536.321078	-1418.43	28.2175	63.535	187.39	125157.49
num_prior_txns	20420.0	16.021009	16.531393	0.00	4.0000	11.000	22.00	172.00
is_fraud	20420.0	0.043879	0.204830	0.00	0.0000	0.000	0.00	1.00

Check. Explicit tail percentiles for **amount**, including the 1st, 5th, 95th and 99th that `describe()` does not report by default. The tails are where outliers live and where the Section 7 thresholds are set.

Finding. The distribution climbs steeply in the upper tail: $p_{25} = 28.22$, $p_{50} = 63.54$, $p_{75} = 187.39$, $p_{95} = 1,002.95$, $p_{99} = 2,478.84$ — yet the observed maximum is 125,157.49, more than fifty times the 99th percentile. The extreme values are concentrated in a very small fraction of rows well beyond p_{99} , which is exactly the profile that distorts mean-based statistics.

CODE

```
df["amount"].quantile([0.01, 0.05, 0.25, 0.5, 0.75, 0.95, 0.99])
```

OUTPUT

```
0.01      4.4119
0.05     9.6595
0.25    28.2175
0.50    63.5350
0.75   187.3900
0.95  1002.9475
```

```
0.99    2478.8433
Name: amount, dtype: float64
```

6. Distribution and skew

Check. Histogram and boxplot of `amount`, both clipped at the 99th percentile *for display only* — without clipping, the extreme tail stretches the axis until the remaining 99% of the data collapses into a single bar. `clip()` returns a new Series; the underlying column is not modified. Skewness is then computed on the unmodified column.

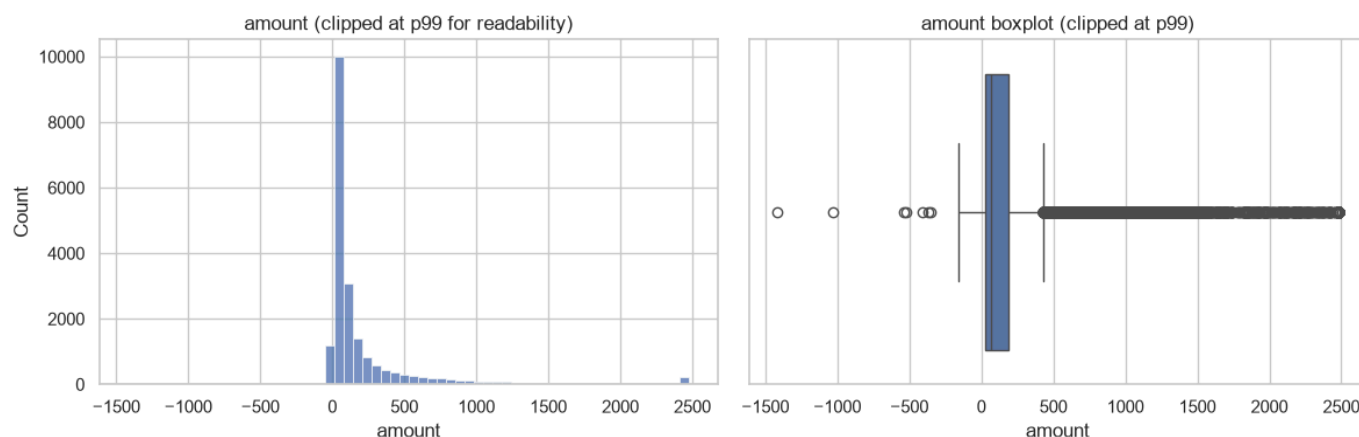
Finding. The histogram shows the expected shape: a dense mass at low values with a long right tail. The boxplot flags a substantial band of points beyond the upper whisker *even after clipping*, which is a first indication that the 1.5xIQR rule will flag a large share of this column. Skewness is **51.49** — an order of magnitude above the 1-3 range typical of naturally skewed real-world spend data, confirming that extreme values, not just natural skew, are driving the shape. Mean 258.84 versus median 63.54 restates the same result numerically.

CODE

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.histplot(df["amount"].clip(upper=df["amount"].quantile(0.99)), bins=60, ax=axes[0])
axes[0].set_title("amount (clipped at p99 for readability)")
sns.boxplot(x=df["amount"].clip(upper=df["amount"].quantile(0.99)), ax=axes[1])
axes[1].set_title("amount boxplot (clipped at p99)")
plt.tight_layout()
plt.show()

print("skew(amount):", df["amount"].skew())
print("mean vs median:", df["amount"].mean(), "vs", df["amount"].median())
```

OUTPUT



```
skew(amount): 51.49214460547081
mean vs median: 258.8420519098923 vs 63.535
```

Check. The same treatment applied to one categorical column (`merchant_category`) and one well-behaved numeric column (`customer_age`), as a reference point.

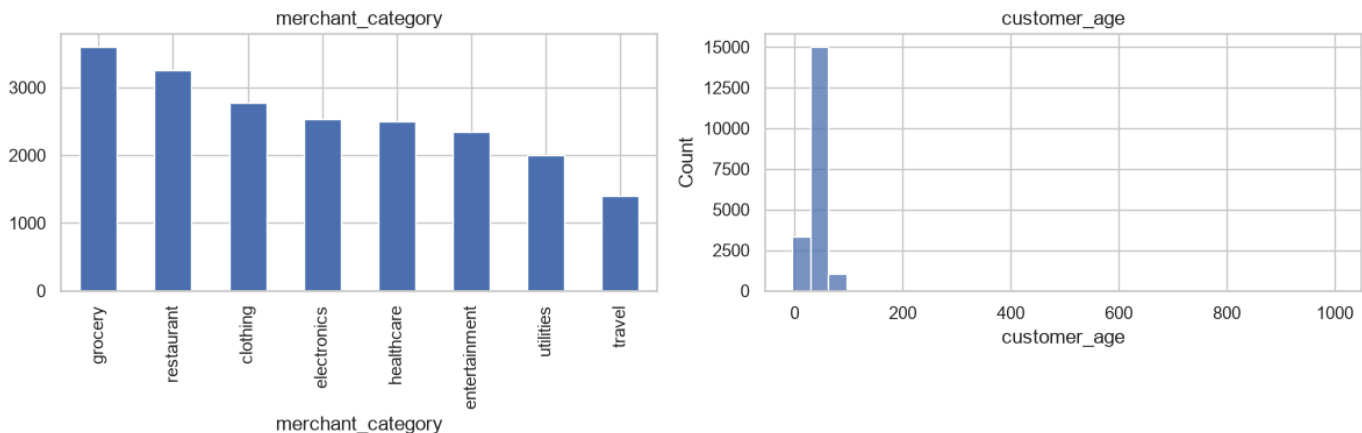
Finding. The 8 merchant categories are broadly balanced, with grocery and restaurant modestly over-represented; there is no dominant or near-empty category that would destabilise a category-level aggregate. `customer_age` is approximately symmetric and centred near 41, in contrast to `amount`.

Implication. Skew is a property of individual columns, not of the table. A blanket "use the median everywhere" policy would be as wrong as a blanket "use the mean" — each column needs to be assessed on its own distribution.

CODE

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
df["merchant_category"].value_counts().plot(kind="bar", ax=axes[0], title="merchant_category")
sns.histplot(df["customer_age"].dropna(), bins=30, ax=axes[1])
axes[1].set_title("customer_age")
plt.tight_layout()
plt.show()
```

OUTPUT



7. Outlier detection

Two standard methods are applied to `amount` and compared, since neither is authoritative on skewed data.

Method 1 - IQR (Tukey) fences. Anything below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ is flagged. **Result: fences of [-210.54, 426.15], flagging 2,742 rows (13.43%).**

Method 2 - z-score. Values more than three standard deviations from the mean are flagged. **Result: 57 rows (0.28%).**

Interpretation of the gap. The two methods disagree by a factor of 48, and the disagreement is itself the finding. The z-score uses the mean and standard deviation, both of which are inflated by the very outliers being searched for — with a standard deviation of 1,536 driven by million-scale

values, ordinary large transactions no longer look unusual, so the method under-reports. The IQR method is robust to that inflation, but on a genuinely skewed column it over-reports: a large legitimate travel or electronics order clears the 426.15 fence without being an error. **Neither figure should be read as "13.43% of rows are wrong."** The defensible reading is that the IQR band defines a review population, not an error population.

Unambiguous defect. Independently of either method, 26 rows carry a **negative amount**. This requires no statistical threshold: the dataset documents no refund or reversal convention, so a negative sale amount is invalid by definition. This is the one outlier finding that can be stated as an error rather than as a candidate for review.

CODE

```
q1, q3 = df["amount"].quantile([0.25, 0.75])
iqr = q3 - q1
lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
iqr_outliers = df[(df["amount"] < lower) | (df["amount"] > upper)]
print(f"IQR fences: [{lower:.2f}, {upper:.2f}] -> {len(iqr_outliers)} outlier rows
({len(iqr_outliers)/len(df)*100:.2f}%)")

z = (df["amount"] - df["amount"].mean()) / df["amount"].std()
z_outliers = df[z.abs() > 3]
print(f"|z| > 3 -> {len(z_outliers)} outlier rows ({len(z_outliers)/len(df)*100:.2f}%)")

# negative amounts are a different, more obvious problem than "extreme but positive"
print("negative amounts:", (df['amount'] < 0).sum())
```

OUTPUT

```
IQR fences: [-210.54, 426.15] -> 2742 outlier rows (13.43%)
|z| > 3 -> 57 outlier rows (0.28%)
negative amounts: 26
```

8. Correlation structure

Check. Pearson correlation across the numeric columns, computed twice: once on the raw data and once on a cleaned subset with negative amounts and the top 1% of `amount` removed. The side-by-side comparison isolates how much the outliers distort the measured relationships.

Finding. In the raw matrix, `amount` correlates only weakly with `unit_price` and `quantity` — a result that contradicts how the field is defined. After removing the extreme tail, `amount` vs `unit_price` rises to roughly **0.84** and `amount` vs `quantity` to roughly **0.2**.

Interpretation. The relationship was always there; the outliers were masking it. Pearson correlation is a variance-based statistic, and a handful of million-scale values contribute so much variance that the genuine signal is diluted toward zero. The practical lesson is that correlations computed on unprofiled data can be wrong in the *conservative* direction — reporting no relationship where a strong one exists — which is the harder error to notice.

`is_fraud` also shows a non-trivial negative correlation with `num_prior_txns`. That relationship is discussed in Section 11, where it serves as the counter-example: a real correlation that is nonetheless not causal.

CODE

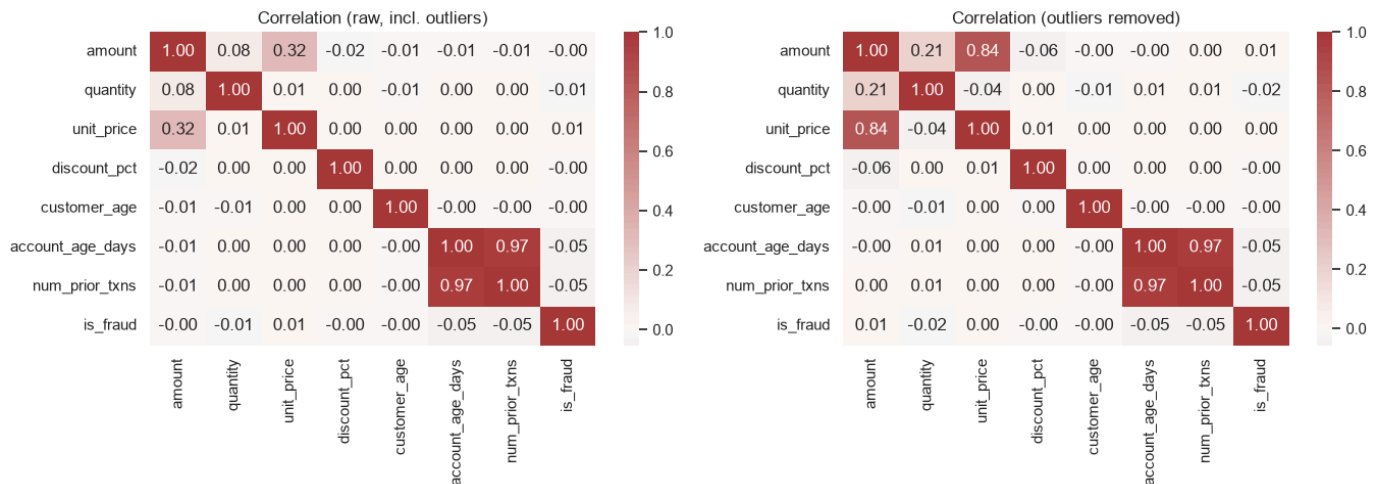
```
num_cols = ["amount", "quantity", "unit_price", "discount_pct", "customer_age",
            "account_age_days", "num_prior_txns", "is_fraud"]

corr_raw = df[num_cols].corr(numeric_only=True)

clean = df[(df["amount"] > 0) & (df["amount"] < df["amount"].quantile(0.99))]
corr_clean = clean[num_cols].corr(numeric_only=True)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
sns.heatmap(corr_raw, annot=True, fmt=".2f", cmap="vlag", center=0, ax=axes[0])
axes[0].set_title("Correlation (raw, incl. outliers)")
sns.heatmap(corr_clean, annot=True, fmt=".2f", cmap="vlag", center=0, ax=axes[1])
axes[1].set_title("Correlation (outliers removed)")
plt.tight_layout()
plt.show()
```

OUTPUT



9. Distributed equivalent (PySpark)

The pandas profile above runs in a single process, which is appropriate at 20,420 rows but does not extend to production volumes. The cell below is the same profile expressed in PySpark, where the work is distributed across a cluster. It is left commented out because it requires an active `SparkSession`, which this notebook does not create — it is normally supplied by the lab environment (EMR or Glue). Column names match the pandas version one-to-one.

Profiling step	pandas (this notebook)	PySpark equivalent
Schema	<code>df.info()</code>	<code>sdf.printSchema()</code>
Descriptive stats	<code>df.describe()</code>	<code>sdf.summary().show()</code>
Null counts	<code>df.isna().sum()</code>	<code>F.count(F.when(F.col(c).isNull(), c))</code> per column
Duplicate rows	<code>df.duplicated()</code>	<code>groupBy(sdf.columns).count().filter("count > 1")</code>
Quantiles	<code>df["amount"].quantile([...])</code>	<code>sdf.approxQuantile("amount", [...], 0.01)</code>
Correlation	<code>df.corr()</code>	<code>sdf.stat.corr("amount", "unit_price")</code>

Two differences are worth noting when migrating. Spark has no single-call equivalent of `isna().sum()`, so the null profile is built as a per-column aggregation. And `approxQuantile` is approximate by design — its third argument is the permitted relative error (0.01 here) — which is the trade Spark makes to compute quantiles on data that does not fit in one machine's memory.

CODE

```
# --- PySpark version (run this in the lab's Spark/Jupyter kernel) ---
# from pyspark.sql import SparkSession
# from pyspark.sql import functions as F
#
# spark = SparkSession.builder.appName("transactions-profile").getOrCreate()
# sdf = spark.read.csv("transactions.csv", header=True, inferSchema=True)
#
# sdf.printSchema()                                # schema
# sdf.summary().show()                             # count/mean/stddev/min/25%/50%/75%/max
# sdf.select([F.count(F.when(F.col(c).isNull(), c)).alias(c) for c in sdf.columns]).show() # nulls
# sdf.groupBy(sdf.columns).count().filter("count > 1").show() # duplicate rows
# sdf.approxQuantile("amount", [0.25, 0.5, 0.75, 0.99], 0.01) # quantiles
# sdf.stat.corr("amount", "unit_price")             # pairwise correlation
```

10. Executable data-quality rules

The findings above are point-in-time observations. This section converts them into eight executable assertions that can be re-run against any future load of the same table and will report `PASS` / `FAIL` automatically.

Design. Each rule is registered through `add_rule(name, description, violation_mask, max_violation_rate)`, where `violation_mask` is a boolean Series marking offending rows. The helper counts violations, converts the count to a **rate** so severity stays comparable across datasets of different sizes, compares that rate to the rule's threshold, and appends the result to a summary table. Most thresholds are 0.0 (zero tolerance); two are set to 0.02 where a small violation rate is expected rather than defective.

#	Rule	Checks	Threshold
1	pk_unique	transaction_id never repeats	0%
2	no_full_duplicates	no row is an exact copy of another	0%
3	required_not_null	customer_id, amount, transaction_ts are never null	0%
4	amount_positive	amount > 0	0%
5	amount_within_iqr_fence	amount inside the Section 7 fences	2%
6	age_plausible	customer_age in [18, 100], nulls excluded	0%
7	category_in_domain	merchant_category in the known 8-value set	0%
8	amount_matches_line_items	amount within 5 units of quantity x unit_price x (1 - discount_pct)	2%

Two design notes. Rule 6 excludes nulls deliberately — a *missing* age is Rule 3's concern, while a *present but impossible* age (999) is this rule's. Rule 5 carries a 2% tolerance precisely because Section 7 established that the IQR fence over-flags on skewed data; the rule still fails here, which is the intended signal that the tail is wider than tolerable rather than that every flagged row is an error.

Result: 2 PASS, 6 FAIL.

Rule	Violations	Rate	Status
pk_unique	840	4.11%	FAIL
no_full_duplicates	800	3.92%	FAIL
required_not_null	0	0.00%	PASS
amount_positive	26	0.13%	FAIL
amount_within_iqr_fence	2,742	13.43%	FAIL
age_plausible	16	0.08%	FAIL
category_in_domain	0	0.00%	PASS
amount_matches_line_items	1,185	5.80%	FAIL

Rule 8 is the most informative of the eight and did not surface anywhere else in the profile: 5.80% of rows carry an amount that cannot be reconstructed from their own line items. Unlike the outlier rules, this is a pure internal-consistency failure — the row disagrees with itself — and it would pass every distributional check in this notebook unnoticed.

A mixed PASS/FAIL result is the correct outcome for this dataset. A rule set that passed everything on data with 400 duplicate rows and 26 negative amounts would only demonstrate that the rules were too weak to detect them.

```

rules = []

def add_rule(name, description, violation_mask, max_violation_rate=0.0):
    n = int(violation_mask.sum())
    rate = n / len(df)
    rules.append({
        "rule": name,
        "description": description,
        "violations": n,
        "violation_rate": round(rate, 4),
        "threshold": max_violation_rate,
        "status": "PASS" if rate <= max_violation_rate else "FAIL",
    })

# 1. Primary key uniqueness
add_rule("pk_unique", "transaction_id must be unique",
        df["transaction_id"].duplicated(keep=False), max_violation_rate=0.0)

# 2. No exact duplicate rows
add_rule("no_full_duplicates", "no row may be a byte-for-byte duplicate of another",
        df.duplicated(keep=False), max_violation_rate=0.0)

# 3. Not-null on required fields
add_rule("required_not_null", "customer_id, amount, transaction_ts must never be null",
        df[["customer_id", "amount", "transaction_ts"]].isna().any(axis=1),
        max_violation_rate=0.0)

# 4. Amount must be positive
add_rule("amount_positive", "amount must be > 0 (a real sale, not a refund/error)",
        df["amount"] <= 0, max_violation_rate=0.0)

# 5. Amount within a plausible range (IQR-based, allows a small tail)
add_rule("amount_within_iqr_fence", "amount must fall inside [Q1-1.5*IQR, Q3+1.5*IQR]",
        (df["amount"] < lower) | (df["amount"] > upper), max_violation_rate=0.02)

# 6. customer_age within a plausible human range
add_rule("age_plausible", "customer_age must be between 18 and 100 (nulls excluded)",
        df["customer_age"].notna() & ((df["customer_age"] < 18) | (df["customer_age"] > 100)),
        max_violation_rate=0.0)

# 7. Categorical domain check
valid_categories = {"grocery", "electronics", "travel", "restaurant", "clothing",
                    "utilities", "entertainment", "healthcare"}

```

```

add_rule("category_in_domain", "merchant_category must be one of the known categories",
        ~df["merchant_category"].isin(valid_categories), max_violation_rate=0.0)

# 8. Referential/consistency check: amount should roughly equal quantity*unit_price*(1-discount_pct)
expected = df["quantity"] * df["unit_price"] * (1 - df["discount_pct"].fillna(0))
add_rule("amount_matches_line_items", "amount should be within 5 units of quantity*unit_price*(1-
discount_pct)",
        (df["amount"] - expected).abs() > 5, max_violation_rate=0.02)

rules_df = pd.DataFrame(rules)
rules_df

```

OUTPUT

	rule	description	violations	violation_rate	threshold	status
0	pk_unique	transaction_id must be unique	840	0.0411	0.00	FAIL
1	no_full_duplicates	no row may be a byte-for-byte duplicate of ano...	800	0.0392	0.00	FAIL
2	required_not_null	customer_id, amount, transaction_ts must never...	0	0.0000	0.00	PASS
3	amount_positive	amount must be > 0 (a real sale, not a refund/...	26	0.0013	0.00	FAIL
4	amount_within_iqr_fence	amount must fall inside [Q1-1.5*IQR, Q3+1.5*IQR]	2742	0.1343	0.02	FAIL
5	age_plausible	customer_age must be between 18 and 100 (nulls...	16	0.0008	0.00	FAIL
6	category_in_domain	merchant_category must be one of the known cat...	0	0.0000	0.00	PASS
7	amount_matches_line_items	amount should be within 5 units of quantity*un...	1185	0.0580	0.02	FAIL

11. Discussion

11.1 Median versus mean on skewed data

`amount` is right-skewed (skew = 51.5), driven by a small number of very large transactions together with injected data-entry outliers. On this column the mean is 258.84 while the median is 63.54.

The mean sits roughly four times above where a typical transaction actually falls, because it is sensitive to magnitude: a single 125,157.49 value pulls the average up as much as two thousand typical transactions do. The median is the 50th percentile and responds only to rank, so that same extreme value shifts it no more than a 200 value would. For "what does a typical transaction look like," the median is the defensible statistic on this column.

The mean is not thereby wrong — it is the correct statistic for a different question. Total revenue reconciliation requires magnitude sensitivity (mean x count = sum), so the mean is the right tool there. The conclusion is not "prefer the median" but **match the statistic to the question**: typical value to the median, total exposure to the mean or sum, with the skew disclosed alongside it. Reporting a mean of 258.84 as "average spend" without that disclosure would materially misrepresent this dataset.

11.2 Correlation does not imply causation

The correlation matrix shows a real negative relationship between `num_prior_txns` and `is_fraud`. Read naively, this suggests that customers with more transaction history commit less fraud, and

that transaction count could serve as a risk signal.

The data generation process contradicts that reading. `is_fraud` is derived from `account_age_days` and `channel`, not from `num_prior_txns`. New accounts are both riskier *and* mechanically have fewer prior transactions, so account age is a **confounder** driving both variables. The correlation is real and reproducible; the causal claim built on it is false.

Four mechanisms produce correlation without causation, all present or plausible here:

1. **Confounding** — a third variable drives both, as above.
2. **Reverse causation** — the arrow may run the other way; fraud on an account can itself suppress future transaction counts.
3. **Coincidence or small-sample noise** — especially with rare events. `is_fraud` is positive on only 4.4% of rows, so correlations involving it rest on roughly 900 positive cases.
4. **Construction artifacts** — `amount` and `unit_price` correlate at ~ 0.84 *because amount was computed from unit_price*. This one is genuinely causal, by construction.

That last case is the point worth carrying forward. Two correlations of similar strength in the same matrix, one causal by construction and one purely confounded, are indistinguishable from the coefficients alone. Separating them required knowledge of how the columns were produced — not more statistics. Establishing causation needs domain knowledge, a controlled comparison, or an experiment.

12. Limitations and recommended next steps

Limitations of this profile.

- The dataset is synthetic, so the defect rates here reflect an injection process rather than a real production pipeline, and should not be treated as a baseline for real data.
- `amount` mixes at least three currencies (MXN, BRL, USD) with no FX rates supplied. Every aggregate statistic on `amount` in this report — mean, median, quantiles, outlier fences — therefore mixes units and would need to be recomputed per currency for any business use.
- `transaction_ts` was validated only for type, not for content. Chronological checks (future dates, gaps in coverage, duplicate timestamps per customer) were out of scope.
- Correlation was assessed with Pearson only, which measures linear association; monotonic but non-linear relationships would not appear.

Recommended next steps, in priority order.

1. **Fix `transaction_id` upstream.** Until the key is unique, no join or aggregation keyed on it is trustworthy. This blocks downstream use and is the highest-priority item.
2. **Investigate the 1,185 amount/line-item mismatches.** Determine whether they originate in rounding, in an undocumented fee or tax component, or in genuine corruption — the fix differs in each case.
3. **De-duplicate the 400 exact duplicate rows** and add the source-system idempotency check that would have prevented them.

4. **Quarantine the 26 negative amounts and 16 impossible ages** for manual review; both are small enough to resolve case by case.
5. **Document the country / currency gap as online-channel-specific** and populate it with an explicit UNKNOWN sentinel rather than imputing.
6. **Schedule the Section 10 rule set** against each incoming load, and treat threshold breaches as pipeline alerts rather than as a report produced after the fact.